

ATIVIDADE PRÁTICA

CAMPO MINADO



Integrando Lista, Pilha, Fila e Conjunto — e medindo o custo de cada jogada

Grau 1 • Aula 4 • Python 3.10+ (ou C11) • Prática de autoestudo, sem entrega • Tempo sugerido: 4 a 6 horas

OBJETIVO Aplicar, em um programa completo, as estruturas das Aulas 1 a 3 — lista contígua, Pilha, Fila e Conjunto — e analisar o custo de cada parte com as ferramentas da Aula 4: contagem de operações, notação O e melhor e pior caso.

SEM ENTREGA Esta é uma atividade de prática e autoestudo: não há entrega nem nota. Uma solução de exemplo, completa e comentada, está disponível no repositório da disciplina, junto com o gabarito das partes sem código. Tente resolver antes de consultá-la: é tentando que se aprende.

1. O jogo e as quatro aulas

O Campo Minado é um jogo de lógica clássico: um tabuleiro de células fechadas esconde algumas minas, e o jogador precisa abrir todas as células seguras usando as pistas numéricas que aparecem. Por trás de um jogo simples, há exatamente o que estudamos no Grau 1: uma estrutura contígua para o tabuleiro, uma Fila (ou uma Pilha) para abrir áreas vazias, um Conjunto para sortear minas sem repetição, uma Pilha para o histórico de jogadas e, sobre tudo isso, a pergunta da Aula 4: *quanto custa cada operação?*

AULA	CONCEITO	ONDE APARECE NO JOGO
1	Lista contígua, acesso por índice em $O(1)$	o tabuleiro inteiro é guardado em listas lineares
2	Fila encadeada (FIFO)	abrir uma área vazia “em ondas”
2	Pilha contígua (LIFO)	abrir “em profundidade” e guardar o histórico de jogadas
3	Conjunto, sem duplicatas	sortear as minas sem repetição e guardar as bandeiras
4	Contagem de operações, O , Ω , Θ , casos	contador de operações, ranking com Insertion Sort e a análise da Parte D

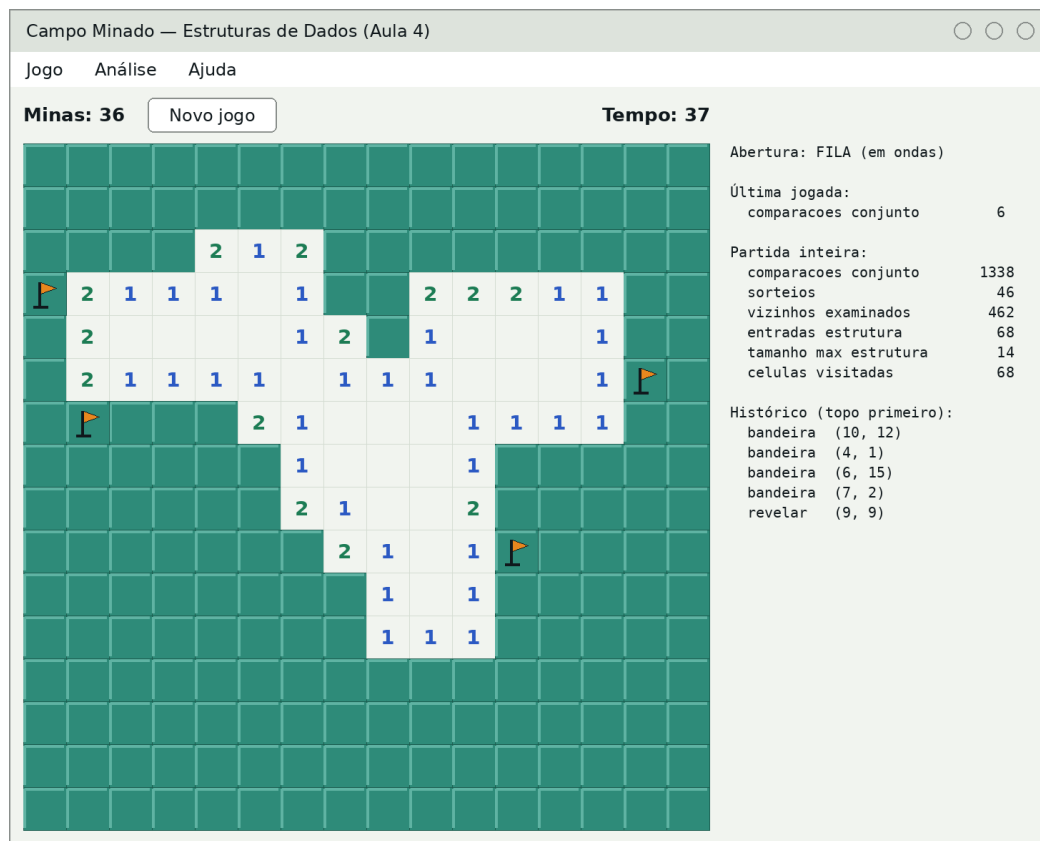


Figura 1. A interface gráfica do jogo (nível Médio), com o painel de análise à direita. Ilustração gerada a partir do código de desenho do próprio jogo.

2. As regras do jogo, em resumo

- O tabuleiro tem L linhas e C colunas de células fechadas; k delas escondem uma mina.
- Revelar uma célula com mina encerra a partida com **derrota**.
- Uma célula segura revelada mostra um **número**: quantas minas existem entre as suas até 8 vizinhas (em cima, embaixo, dos lados e nas diagonais).
- Uma célula com **0** não tem minas em volta; por isso o jogo abre automaticamente as suas vizinhas — e as vizinhas das vizinhas que também forem 0. É a **abertura em cascata**.
- O jogador pode marcar uma **bandeira** onde acha que há uma mina. Células com bandeira não podem ser reveladas (nem pela cascata) até a bandeira ser retirada.
- O **primeiro clique é sempre seguro**: as minas só são sorteadas depois dele, longe da célula clicada e das suas vizinhas.
- A partida termina com **vitória** quando todas as células seguras estão abertas.

NÍVEL	TABULEIRO	MINAS
Fácil	9×9	10
Médio	16×16	40
Difícil	16×30	99



NA APOSTILA O capítulo 14 da apostila da Aula 4, “Campo Minado: regras do jogo e técnicas de implementação”, traz as regras em detalhes, com exemplos passo a passo, e explica cada algoritmo e técnica usados nesta atividade.

3. Arquivos e regras

Os arquivos estão na pasta `aula04-complexidade` do repositório da disciplina, <https://github.com/ProfessorFilipo/EstruturasDados>:

```
aula04-complexidade/
├── exercicios/campo_minado/    <- ESQUELETO: comece por aqui
│   ├── tabuleiro.py           <- TODOs 1 a 10 (a lógica do jogo)
│   ├── ranking.py             <- TODOs 11 e 12
│   ├── estruturas.py          <- Pilha, Fila e Conjunto (prontos)
│   ├── jogo_gui.py            <- interface gráfica (pronta)
│   ├── jogo_terminal.py       <- interface de texto (pronta)
│   ├── testes.py              <- testes automáticos (prontos)
│   ├── analise_complexidade.py <- medições da Parte D (pronto)
│   └── campo_minado_arquivo_unico.py <- tudo em um arquivo, para IDE online
└── solucoes/campo_minado/     <- SOLUÇÃO DE EXEMPLO e GABARITO.md
```

- Complete somente os trechos marcados com `TODO`. Não mude nomes nem parâmetros dos métodos: as interfaces e os testes dependem deles.
- Em `tabuleiro.py`, não use `set()`, `collections.deque` nem `queue.Queue`: use a `Pilha`, a `Fila` e o `Conjunto` de `estruturas.py` — as mesmas das Aulas 2 e 3.
- Use só a biblioteca padrão do Python (nada de `pip install`).
- No código, linhas e colunas começam em 0. As interfaces mostram a partir de 1 para o jogador: a célula que ele chama de (5, 1) é a (4, 0) no código.

4. Preparando o ambiente

4.1 Python e PyCharm, passo a passo (Windows)

- 1. Instale o Python.** Em python.org/downloads, baixe a versão mais recente do Python 3. Na primeira tela do instalador, marque a opção **Add python.exe to PATH** e clique em **Install Now**. A instalação padrão já inclui o `tkinter`, usado pela interface gráfica.
- 2. Confira a instalação.** Abra o Prompt de Comando e digite `py --version`. Deve aparecer algo como `Python 3.12.x`.
- 3. Instale o PyCharm.** Em jetbrains.com/pycharm/download, baixe e instale o PyCharm. As funções essenciais são gratuitas e bastam para esta atividade.
- 4. Baixe os arquivos.** Na página do repositório no GitHub, clique no botão verde **Code** e depois em **Download ZIP**. Descompacte o arquivo em uma pasta sua.
- 5. Abra a pasta do esqueleto.** No PyCharm, use **File > Open** e escolha a pasta `aula04-complexidade/exercicios/campo_minado`.

6. **Confira o interpretador.** No canto inferior direito da janela, o PyCharm mostra a versão do Python em uso. Se aparecer “No interpreter”, clique ali e escolha o Python que você instalou.
7. **Rode o jogo.** Clique com o botão direito em `jogo_gui.py` e escolha **Run**. A janela abre; ao clicar numa célula, aparece o aviso “Falta implementar: TODO 1”. É por aí que você começa.
8. **Rode os testes.** Da mesma forma, rode `testes.py`. Faça isso depois de cada TODO para conferir o que já está certo.

4.2 Outras opções

- **Thonny** (thonny.org): um editor gratuito e bem simples, que já vem com o Python e o tkinter. Boa opção para computadores mais modestos: abra o arquivo e tecle **F5** para rodar.
- **Só IDE online:** use `campo_minado_arquivo_unico.py` no onlineide.pro/playground/python. É a versão de texto, com todo o código em um único arquivo. A interface gráfica não funciona em IDEs online.
- **macOS:** o Python do python.org já traz o tkinter; use `python3` no Terminal. O clique direito do jogo também funciona com **Control + clique**.
- **Linux:** se aparecer `No module named 'tkinter'`, instale o pacote do sistema (no Ubuntu/Debian: `sudo apt install python3-tk`).
- **Linguagem C:** o CLion (gratuito para uso não comercial, como estudo) ou o Code::Blocks (escolha o instalador que já inclui o compilador MinGW). Veja a seção 10.

5. Parte A — Modelagem no papel, antes do código

Resolva esta parte com lápis e papel. Ela é a base de tudo o que você vai programar depois, e o `testes.py` usa exatamente este tabuleiro.

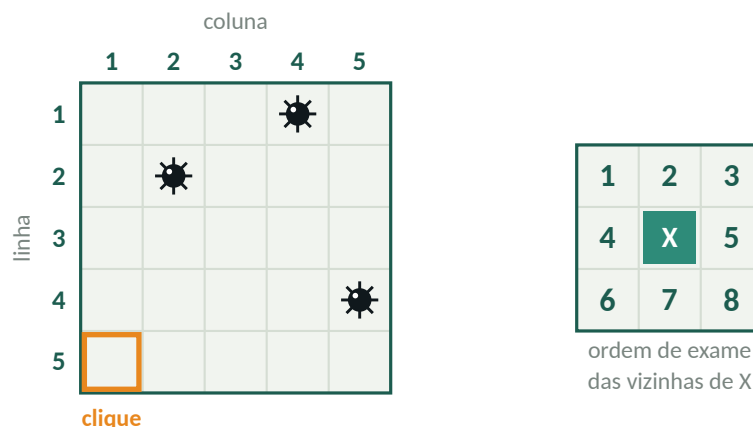


Figura 2. À esquerda, o tabuleiro da Parte A: 5×5 , com minas em (1, 4), (2, 2) e (4, 5), e o clique em (5, 1). À direita, a ordem em que as vizinhas devem ser examinadas: noroeste, norte, nordeste, oeste, leste, sudoeste, sul e sudeste.

- A1. Números.** Desenhe o tabuleiro e escreva, em cada célula segura, quantas minas há entre as suas vizinhas. Atenção às bordas e aos cantos: uma célula de canto tem só 3 vizinhas; uma de borda, 5.
- A2. Índices.** O tabuleiro será guardado em uma lista linear de 25 posições. Contando linhas e colunas a partir de 1, a célula (linha, coluna) fica na posição $\text{índice} = (\text{linha} - 1) \times 5 + (\text{coluna} - 1)$. Calcule o



índice de (1, 4), (2, 2) e (4, 5). Depois faça o caminho inverso: qual célula está no índice 12?

- A3. Abertura com Fila.** Simule o clique em (5, 1) usando uma Fila. Comece colocando (5, 1) na fila e, a cada passo, retire a célula da frente; se o número dela for 0, coloque na fila as vizinhas ainda fechadas, na ordem da figura. Registre cada passo:

PASSO	SAI DA FILA	NÚMERO	ENTRAM NA FILA	FILA DEPOIS DO PASSO
0	—	—	(5,1)	(5,1)
1	(5,1)	0	(4,1) (4,2) (5,2)	(4,1) (4,2) (5,2)
2	...			
...				

- A4. Abertura com Pilha.** Repita a simulação trocando a Fila por uma Pilha (a célula que sai é sempre a do topo). As células abertas são as mesmas? E a ordem?

- A5. Placar.** Quantas células foram abertas pelo clique? Quantas células seguras ainda faltam abrir para vencer?

ATENÇÃO Marque cada célula como aberta **no momento em que ela entra** na estrutura, e não quando ela sai. Assim nenhuma célula entra duas vezes — e você vai medir, na Parte D, quanto trabalho isso economiza.

CHECKPOINT O clique abre 12 células — as mesmas com Fila e com Pilha — e restam 10 células seguras fechadas. Com a Fila, as primeiras a sair são (5, 1), (4, 1), (4, 2) e (5, 2). As tabelas completas estão no `GABARITO.md`, na pasta da solução.

6. Parte B — O núcleo do jogo (TODOs 1 a 7)

Todos estes TODOs ficam em `tabuleiro.py`. Leia o comentário de cada método: ele diz o que fazer, quais contadores atualizar e qual é o custo esperado.

TODO	MÉTODO	O QUE FAZ	CUSTO
1	<code>indice()</code>	(linha, coluna) → posição na lista	$O(1)$
2	<code>coordenadas()</code>	posição na lista → (linha, coluna)	$O(1)$
3	<code>vizinhos()</code>	as até 8 vizinhas, dentro do tabuleiro	$O(1)$
4	<code>_posicionar_minas()</code>	sorteio sem repetição, protegendo o 1º clique	$O(k^2)$
5	<code>_calcular_numeros()</code>	quantas minas há em volta de cada célula	$O(k)$
6	<code>_abrir_a_partir_de()</code>	abertura em cascata com Fila ou Pilha	$O(L \cdot C)$
7	<code>_revelar()</code>	regras da jogada: derrota, abertura, vitória	—

**TODOs 1 e 2 — endereçamento [Aula 1]**

Com colunas contadas a partir de 0: $\text{indice} = \text{linha} * \text{colunas} + \text{coluna}$. Para voltar, use a divisão inteira e o resto: $\text{linha} = \text{indice} // \text{colunas}$ e $\text{coluna} = \text{indice} \% \text{colunas}$. Com 5 colunas, $(1, 0) \rightarrow 5$ e $12 \rightarrow (2, 2)$.

TODO 3 — vizinhas

Percorra a tupla `DIRECOES`, que já está no arquivo, na ordem em que ela aparece (os testes conferem a ordem). Para cada deslocamento, calcule a linha e a coluna vizinhas e guarde o índice só se $0 \leq l < \text{linhas}$ e $0 \leq c < \text{colunas}$. Sem essa verificação, a célula da última coluna “enxergaria” a primeira coluna da linha de baixo.

TODO 4 — sorteio das minas [Aula 3]

Monte um `Conjunto` com as células proibidas (a clicada e suas vizinhas). Depois, enquanto o Conjunto de minas tiver menos que `num_minas` elementos, sorteie um índice com `self._gerador.randrange(self.total_celulas)`, conte em `"sorteios"` e insira-o se não for proibido — o Conjunto já ignora repetições. No fim, faça `self.minas_posicionadas = True` e chame `self._calcular_numeros()`.

TODO 5 — números [Aula 4]

Não pergunte a cada célula “quantas vizinhas são minas?": isso custaria $O(L \cdot C \cdot k)$. Faça o contrário: percorra só as minas (`self.minas.valores()`) e some 1 no número de cada vizinha. São no máximo 8 passos por mina, ou seja, $O(k)$.

TODO 6 — abertura em cascata [Aula 2]

Em Python, funções também são valores: guarde em variáveis as operações de colocar e retirar da estrutura escolhida, e o resto do algoritmo fica igual para Fila e Pilha.

```

usar_fila = self.estrategia == "fila"
pendentes = Fila() if usar_fila else Pilha(self.total_celulas)
colocar = pendentes.enqueue if usar_fila else pendentes.empilhar
retirar = pendentes.dequeue if usar_fila else pendentes.desempilhar

```

A ideia do algoritmo, em português:

```

marque 'inicio' como revelada e coloque na estrutura
enquanto a estrutura não estiver vazia:
    retire uma célula e acrescente-a à lista 'ordem'
    se o número dela for 0:
        para cada vizinha ainda fechada e sem bandeira:
            marque como revelada e coloque na estrutura
    devolva 'ordem'

```

Não esqueça os contadores descritos no comentário do método. O teste `test_cada_celula_entra_uma_unica_vez` confere que, num tabuleiro 12×12 sem minas, exatamente 144 células entram na estrutura.



TODO 7 — a jogada “revelar”

Siga as regras na ordem do comentário: ignore células abertas ou com bandeira; sorteie as minas se for a primeira jogada; registre ("revelar", `linha`, `coluna`) no histórico; se for mina, derrota; senão, abra em cascata e, se `celulas_seguras_restantes` chegar a 0, vitória.

7. Parte C — Bandeiras, histórico e ranking (TODOs 8 a 12)

- **TODO 8 — `_alternar_bandeira()`:** marca a bandeira ou a retira, se já existir (Conjunto: `pertence`, `inserir`, `remover`). Não vale em célula aberta. Registre ("bandeira", `linha`, `coluna`) no histórico.
- **TODO 9 — `_desfazer()`:** consulte o **topo** da Pilha. Se for uma bandeira, desempilhe e desfça; se for um “revelar”, não faça nada — abrir uma célula não tem volta. É o comportamento LIFO: só o topo é acessível.
- **TODO 10 — `ultimas_jogadas()`:** devolva as jogadas mais recentes, do topo para a base, **sem** remover nada da Pilha.
- **TODO 11 — `inserir_ordenado()`,** em `ranking.py`: coloque o novo recorde no fim da lista e desloque para a direita os anteriores com tempo **maior** — exatamente o laço interno do Insertion Sort. Em caso de empate, o recorde mais antigo fica na frente.
- **TODO 12 — `ordenar_por_insercao()`:** o Insertion Sort completo, usado ao carregar o arquivo do ranking. Como o próprio jogo salva o arquivo já ordenado, esse é o **melhor caso** do algoritmo, visto em aula.

8. Parte D — Análise de complexidade

Quando todos os testes passarem, rode `analise_complexidade.py` e use a saída para responder. Use L para linhas, C para colunas, k para minas e f para bandeiras.

- D1.** Qual é o custo de `indice()` e de `coordenadas()`? Por quê?
- D2.** Qual é o custo de `vizinhos()`? Ele depende do tamanho do tabuleiro?
- D3.** Compare as duas formas de calcular os números, preenchendo a tabela abaixo com a saída do programa. Qual é o custo de cada uma, em notação O ? Quantas vezes a forma ingênua é mais cara no nível Difícil?
- D4.** O sorteio das minas insere cada mina em um Conjunto implementado sobre uma lista. Mostre que o total de comparações cresce como $k^2/2$ (dica: soma de Gauss). Que alternativa evitaria as repetições do sorteio, e quando ela valeria mais a pena?
- D5.** Por que a célula é marcada ao **entrar** na estrutura, e não ao sair? Compare as duas colunas da seção 2 da análise.
- D6.** Fila e Pilha abrem as mesmas células? Na mesma ordem? Com o mesmo custo de tempo? E de memória — compare os tamanhos máximos da seção 1 da análise.
- D7.** Qual é o custo de verificar a vitória com o contador `celulas_seguras_restantes`? E se, a cada jogada, varrêssemos a lista `reveladas`?



- D8.** As bandeiras ficam em um Conjunto. Qual é o custo de `pertence()` nele, e como isso afeta a abertura em cascata no pior caso? Como deixar essa consulta em $O(1)$, e qual é o preço dessa troca?
- D9.** Qual é o melhor e o pior caso de `inserir_ordenado()`? Em que caso do Insertion Sort cai o `ordenar_por_insercao()` ao carregar o arquivo salvo pelo próprio jogo?
- D10.** Resuma o custo de uma partida: o primeiro clique, cada clique seguinte, cada bandeira e cada redesenho do tabuleiro.

NÍVEL	$L \times C$	k	PERCORRENDO MINAS	PERGUNTANDO A CADA CÉLULA
Fácil	9×9	10		
Médio	16×16	40		
Difícil	16×30	99		

9. Parte E — Interface gráfica e desafios opcionais

A interface gráfica (`jogo_gui.py`) já vem pronta e usa a sua classe `Tabuleiro`. Clique esquerdo revela; clique direito (ou Control + clique, no Mac) marca bandeira; **Ctrl+Z** desfaz a última bandeira; **F2** começa um novo jogo. Experimente:

- No menu **Análise**, escolha **Abrir com Pilha** e ative **Numerar a ordem de abertura**. Clique numa área vazia e compare com a Fila.
- Observe o **painel de análise**: o primeiro clique custa muito mais que os outros. Por quê?
- Marque uma bandeira e tecle Ctrl+Z. Depois revele uma célula e tente de novo.

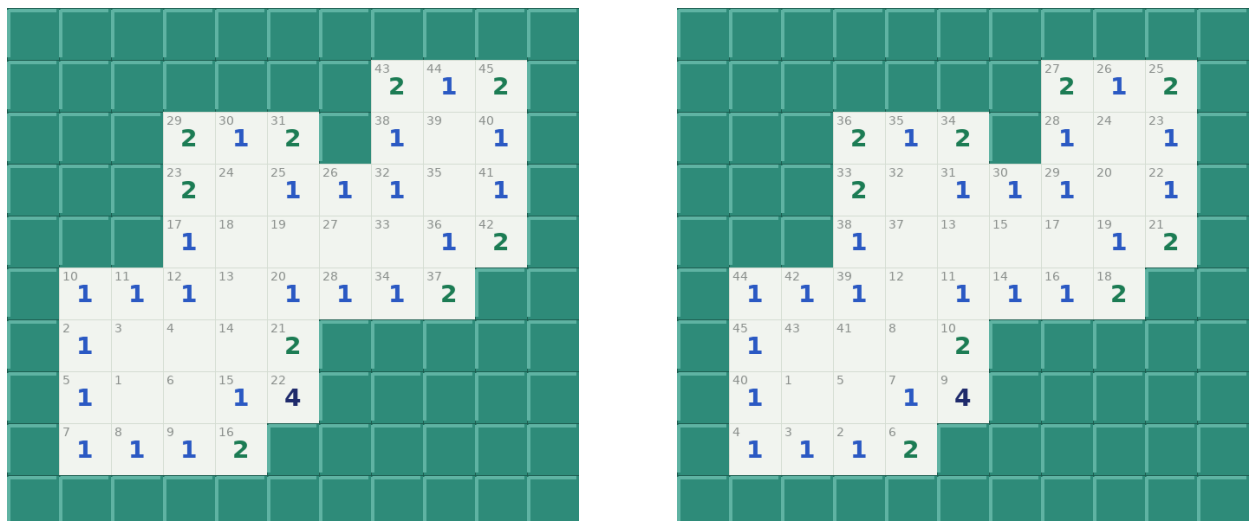


Figura 3. Mesmo tabuleiro, mesmo clique: à esquerda com Fila, à direita com Pilha. Os números pequenos mostram a ordem em que as células foram abertas.

Desafios opcionais, para quem quiser ir além:

- **Abertura por acorde:** clicar num número já aberto abre as vizinhas, se a quantidade de bandeiras em volta for igual ao número.



- **Bandeiras em $O(1)$:** troque o Conjunto de bandeiras por uma lista de booleanos e meça a diferença com o `analise_complexidade.py`.
- **Sorteio por embaralhamento (Fisher-Yates):** implemente e compare o número de comparações com o sorteio atual, principalmente com muitas minas.
- **Nível personalizado** na interface gráfica.

10. Para quem escolher a linguagem C

Em C, faça a versão de terminal, com as mesmas Partes A a D, e use a solução em Python como referência da lógica. Reaproveite as estruturas das aulas anteriores: a Pilha de `aula02-pilhas-filas/c/pilha.c`, a Fila encadeada de `fila.c` (lembre-se do `free` ao desenfileirar) e o Conjunto de `aula03-conjuntos/c/demo_conjunto.c`, aumentando a capacidade para caber o tabuleiro. Uma organização possível:

```
#define MAX_CELULAS (16 * 30)          /* maior tabuleiro: nível Difícil */

typedef struct {
    int linhas, colunas, num_minas, total;
    int numeros[MAX_CELULAS];          /* [Aula 1] lista contígua */
    int reveladas[MAX_CELULAS];        /* 1 = aberta, 0 = fechada */
    Conjunto minas;                    /* [Aula 3] */
    Conjunto bandeiras;                /* [Aula 3] */
    Pilha historico;                   /* [Aula 2] jogadas codificadas */
    int estado;                        /* JOGANDO, VITORIA ou DERROTA */
    int seguras_restantes;             /* vitória em  $O(1)$  */
} Tabuleiro;
```

- Uma jogada cabe em um único int: `jogada = indice * 2 + e_bandeira`. Para decodificar: `indice = jogada / 2` e `e_bandeira = jogada % 2`.
- Sorteio: `srand(time(NULL))` uma vez no início e `rand() % total` para cada candidata. Para testar, use uma semente fixa, como `srand(42)`.
- Leitura de comandos: leia a letra com `scanf(" %c", &comando)`; se for `r` ou `b`, leia a posição com `scanf("%d %d", &linha, &coluna)`.
- Compile sem avisos, como na Aula 2: `cc -std=c11 -Wall -Wextra -pedantic campo_minado.c -o campo_minado`.



11. Casos-limite que o seu código precisa aguentar

SITUAÇÃO	COMPORTAMENTO ESPERADO	O QUE VERIFICA
Primeiro clique num canto ou numa borda	abre uma área, nunca é mina	proteção do 1º clique
Tabuleiro sem minas	o primeiro clique abre tudo e vence	cascata no pior caso
Revelar célula com bandeira	nada acontece	regra das bandeiras
Revelar célula já aberta	nada acontece (devolve lista vazia)	jogada sem efeito
Bandeira dentro de uma área vazia	a cascata contorna a bandeira	cascata respeita bandeiras
Desfazer com um “revelar” no topo	nada acontece	disciplina LIFO
Bandeiras demais	o placar fica negativo	placar = minas — bandeiras
Arquivo de ranking inexistente ou com linhas defeituosas	o jogo abre com ranking vazio ou ignora as linhas ruins	robustez

12. Como conferir sozinho

Rode `testes.py` depois de cada TODO. No começo, quase todos os testes falham, e a mensagem de erro (`NotImplementedError`) indica qual TODO falta. Conforme você avança, eles passam a mostrar `ok`. Os TODOs 4 e 5 são testados juntos (o sorteio termina chamando o cálculo dos números), e o mesmo vale para o 6 e o 7.

TODOS PRONTOS	—	1	1-2	1-3	1-5	1-7	1-8	1-9	1-10	1-11	1-12
Testes ok (de 29)	3	4	6	8	12	19	23	24	25	27	29

CrITÉRIOS de conclusão:

- ☐ As tabelas da Parte A conferem com o gabarito.
- ☐ Os 29 testes de `testes.py` passam.
- ☐ O jogo funciona do começo ao fim, com vitória e derrota, nas duas interfaces.
- ☐ A Fila abre em ondas e a Pilha em profundidade, abrindo as mesmas células.
- ☐ Desfazer só funciona quando a última jogada foi uma bandeira.
- ☐ Você respondeu às questões da Parte D usando a saída de `analise_complexidade.py`.



13. Questões de reflexão

1. Por que guardar o tabuleiro em uma lista linear, e não em uma lista de listas? As duas funcionam — o que a lista linear deixa mais evidente?
2. Por que as regras do jogo ficam em `tabuleiro.py`, separadas das interfaces? O que isso permite fazer?
3. O que aconteceria com o custo da abertura em cascata se a Fila não guardasse a referência para o fim? (É a questão de reflexão da Aula 2, agora com um exemplo real.)

14. A solução de exemplo

A implementação completa está em `solucoes/campo_minado/`. Nela, cada trecho que era um `TODO` aparece marcado com `# TODO n (resolvido)`, o que facilita comparar com a sua versão. O `README.md` dessa pasta sugere uma ordem de leitura, e o `GABARITO.md` traz as respostas comentadas das Partes A e D e das questões de reflexão.

A. Referência rápida da classe `Tabuleiro`

ATRIBUTO OU MÉTODO	PARA QUE SERVE
<code>numeros, reveladas</code>	listas lineares com uma posição por célula
<code>minas, bandeiras</code>	Conjuntos de índices
<code>historico</code>	Pilha de jogadas (<code>acao</code> , <code>linha</code> , <code>coluna</code>)
<code>estado</code>	<code>JOGANDO</code> , <code>VITORIA</code> ou <code>DERROTA</code>
<code>estrategia</code>	"fila" ou "pilha"
<code>contador, contador_partida</code>	operações da última jogada e da partida inteira
<code>revelar(l, c)</code>	jogada revelar; devolve os índices abertos, na ordem
<code>alternar_bandeira(l, c)</code>	marca ou desmarca bandeira; devolve <code>True</code> se aceitou
<code>desfazer()</code>	desfaz a bandeira do topo; devolve <code>(l, c)</code> ou <code>None</code>
<code>ultimas_jogadas(q)</code>	as <code>q</code> jogadas mais recentes, do topo para a base
<code>mapa_visivel()</code>	o que desenhar em cada célula (já pronto)
<code>definir_minas(indices)</code>	coloca minas em posições escolhidas (usado nos testes)



B. Exemplo de sessão na interface de texto

```
Nivel (1, 2 ou 3): 1

Novo jogo: Facil. Digite ? para ver os comandos.

      1 2 3 4 5 6 7 8 9
1  # # # # # # # # #
...
> r 5 5

      1 2 3 4 5 6 7 8 9
1  . 1 # # # # # # #
2  . 1 # # # # # # #
3  . 1 1 2 1 1 # # #
4  . . . . 1 # # #
5  . . . . 1 # # #
6  . . . . 1 # # #
7  . . . . 1 1 # # #
8  . . . . 1 # # #
9  . . . . 1 # # #

Minas restantes: 10      Abertura em cascata: FILA
> b 1 5
(o tabuleiro volta a ser desenhado, agora com um F na linha 1, coluna 5)
> h
Ultimas jogadas (a mais recente primeiro):
bandeira  linha 1, coluna 5
revelar   linha 5, coluna 5
> d
Bandeira desfeita na linha 1, coluna 5.
```

Símbolos: # fechada, F bandeira, . vazia (0), 1 a 8 minas em volta, * mina, X a mina que explodiu, ! bandeira no lugar errado.